

Reviewer Pack — Minimal Rank of Brauer Groups Bounds XOR Circuit Weights

Entry 3ebfe59d6a54 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 17:56:41 UTC

Minimal Rank of Brauer Groups Bounds XOR Circuit Weights

Entry ID: 3ebfe59d6a54 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 17:56:41 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Brauer Group) **Field B** (complexity object): Complexity Theory: Boolean Function Complexity

Statement:

For any given boolean function f with n inputs, the minimal rank of its associated Brauer group $\rho(f)$ is at least $\Omega(2^n/|f|)$, where $|f|$ is the number of non-zero entries in f 's truth table.

Rationale (proposer's reasoning):

The Brauer group captures algebraic information about vector spaces over fields, which could potentially encode structural complexity of boolean functions. A lower bound on $\rho(f)$ would imply a corresponding lower bound on the circuit complexity of f , since each non-zero entry could correspond to a distinct irreducible polynomial in the representation ring.

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7f9d8d7f1a122058

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all boolean functions f with n inputs and non-zero entries $\leq n$, the minimal rank of the Brauer group $\rho(f)$ is at least $\Omega(2^n/|f|)$, where $|f|$ is the number of non-zero entries in f 's truth table. The criterion is falsified if there exists a function f with n inputs and non-zero entries $\leq n$ for which the minimal rank of $\rho(f)$ is less than $\Omega(2^n/|f|)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Brauer Group" AND "Boolean function complexity" - "Minimal rank" AND "XOR circuit weights" AND "Representation theory" - "Truth table" AND "Brauer group" AND "circuit complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=189.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def truth_table(f):
        n = len(f)
        tt = []
        for i in range(2**n):
            inputs = [(i >> j) & 1 for j in range(n)]
            output = f[:]
            for j, val in enumerate(inputs):
                if output[j] == -1:
                    output[j] = val
            tt.append(output)
        return tt

    def matrix_multiplication(A, B):
        m = len(A)
        n = len(B[0])
        p = len(B)
        C = [[0 for _ in range(n)] for _ in range(m)]
        for i in range(m):
            for j in range(n):
                for k in range(p):
                    C[i][j] += A[i][k] * B[k][j]
        return C
```

```

def gaussian_elimination(A):
    m = len(A)
    n = len(A[0])
    rank = 0
    for i in range(m):
        if i < n:
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] != 0:
                for j in range(n-1, i-1, -1):
                    A[i][j] /= A[i][i]
                for j in range(m):
                    if j != i and A[j][i] != 0:
                        for k in range(n-1, i-1, -1):
                            A[j][k] -= A[j][i] * A[i][k]
            rank += 1
    return rank

def minimal_rank(f):
    n = len(f)
    tt = truth_table(f)
    m = len(tt)
    A = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            if tt[i][j] == 1:
                A[i][j] = 1
            elif tt[i][j] == 0:
                A[i][j] = -1

    rank_value = gaussian_elimination(A)
    return rank_value

def count_non_zero_entries(f):
    return sum(1 for x in f if x != 0)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    for _ in range(5): # Ensure at least 5 instances per size
        f = generate_boolean_function(n)
        rank_value = minimal_rank(f)
        non_zero_entries = count_non_zero_entries(f)
        if non_zero_entries == 0:
            continue
        results.append((rank_value, non_zero_entries))

if not results:
    return {
        "metric_name": "minimal_rank",
        "metric_value": 0,
        "instances_tested": 0,
        "conjecture_holds": False,
    }

```

```

        "counterexample": "mapping_undefined"
    }

    total_rank = sum(result[0] for result in results)
    total_entries = sum(result[1] for result in results)
    mean_rank = Fraction(total_rank, len(results))
    lower_bound = Fraction(2*n / total_entries, 1)

    return {
        "metric_name": "minimal_rank",
        "metric_value": float(mean_rank),
        "instances_tested": len(results),
        "conjecture_holds": mean_rank >= lower_bound,
        "counterexample": "" if mean_rank >= lower_bound else f"mean_rank={mean_rank} < lower_bound"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    total_rank = sum(result["metric_value"] * result["instances_tested"] for result in results)
    total_instances = sum(result["instances_tested"] for result in results)
    mean_rank = Fraction(total_rank, total_instances)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0 support_fraction=1")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify the conjecture’s support or falsification. | next: Re-run the test with a different boolean function to gather data for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10395
2	preregistration	ollama_remote	glm4:latest	0	0	6597
3	novelty	ollama_remote	glm4:latest	0	0	4656
4	novelty	ollama_remote	glm4:latest	0	0	5606
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41935
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12280
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10103
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14033
9	judge	ollama_remote	glm4:latest	0	0	62024

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 167628 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/3ebfe59d6a54.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3ebfe59d6a54.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3ebfe59d6a54.tar.gz (if generated)